

Herald



Herald — Issues

Field	Value
Revision	15
Created	2026-05-20
Last modified	2026-05-22
Status	active
Status summary	r15 captures Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure; tag v0.4.0 deferred to T13b post live evidence). HRD-100 atomic Issues→Fixed in this revision. New Cobra subcommand <code>pherald listen</code> runs the closed-loop runtime: Telegram <code>getUpdates</code> long-poll (<code>OnText/OnPhoto/OnDocument/OnVoice</code>) + bot self-filter (<code>bot.Me.Username</code> anti-echo guard, fails <code>Subscribe</code> if <code>getMe</code> returned no <code>username</code>) + sha256 content-addressed attachment download into <code>~/ .herald/ inbox/<sha256>.<ext></code> + Claude Code dispatch with Opus pinned in argv (<code>claude --model claude-opus-4-7 --resume <UUID> --print <envelope></code>) + envelope pre-text (verbatim operator wording "We have received new message from our communication channel") preceding the <code><<<HERALD-DISPATCH-v1>>></code> structured block + <code><<<HERALD-REPLY>>></code> parser with action routing (<code>reply / issue.open / event.emit</code>) + <code>tgram.Adapter.SendReply</code> wiring <code>reply_to_message_id</code> from the inbound <code>message_id</code>. T10a --qa-out-dir flag journals every inbound/outbound event as JSONL per §107.x. 12 commits (T1=ad87d7f..T12=96c7c6b). 8 new e2e invariants E63-E70 added (currently SKIP with documented reasons — converting to PASS at T10b when operator runs the live closed-loop and commits <code>docs/qa/HRD-100/</code>). Wave 6 mutation gate <code>tests/test_wave6_mutation_meta.sh</code> lands 3 paired hermetic mutations. Tag v0.4.0 deferred to T13b (operator-supplied live evidence per §107.x). Prior r14 captured Wave 4b TOON <code>application/toon</code> content negotiation (tag v0.3.0). Prior r13 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3). r12 closed HRD-016 (Wave 3b pherald Runner live). r11 closed HRD-011 (Telegram live, <code>message_id=5</code>).
Issues	HRD-008, HRD-015, HRD-018..HRD-027, HRD-029..HRD-056, HRD-081, HRD-085..HRD-090
Issues summary	47 open / in-progress workable items. r15: HRD-100 closed atomically (Wave 6 pherald inbound runtime — code-doc closure; live evidence operator-pending → T10b). r14: substrate upgrade only (Wave 4b TOON; no HRDs migrated). r12:

Field	Value
	HRD-016 closed atomically (Wave 3b pherald Runner live + e2e E37-E42 PASS or honest SKIP-with-reason when PG :24100 absent). r11: HRD-011 (Telegram live) closed atomically with live message_id=5 evidence. r10 doc-cleanup renumbered HRD-093 → HRD-099. r9: HRD-028 + HRD-098 closed (Wave 3a). Carry-over to Wave 3c: HRD-024 (iherald paging), HRD-033 (destructive-guard body), HRD-018..027 flavor-specific bindings, plus the rest of the §43 command catalogue.
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-028, HRD-080, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099, HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114
Fixed summary	see Fixed.md.
Continuation	see CONTINUATION.md.

Table of contents

- [Open](#)
- [In progress](#)
- [Blocked](#)
- [Conventions](#)

Open

Per Universal §11.4.74 every new row carries a Catalogue-Check line in its References cell (one of reuse|extend|no-match <org/repo>@<sha>). For HRDs whose catalogue-check is still pending, the row records no-match (2026-05-20) — operators reviewing the implementation PR will re-verify.

ID	Type	Status	Criticality	Title
HRD-015	task	open	low	Inheritance gate I8 invariants for Go scaffold (go.work + commons/typ test passes)
HRD-018	task	in_progress	high	commons_constitution Go package: Evaluator interface + 12 ev + constitution_state + constitution_bindings table migrations + bundle mode-ladder runtime config
HRD-019	task	open	high	cherald constitution bindings — bulk implementation: ~30 .poli + .gate.failed/.gate.recovered/.credential.leak/. handlers; /v1/compliance pull surface
HRD-020	task	open	high	sherald host-safety + repo-safety bindings (§9.1/.2/.3, §12.1/.2/.3/.6 §11.4.41, §11.4.71) — destructive-op detection hook, force-push intero watcher
HRD-021	task	open	middle	bherald CI/test bindings (§1, §11.4.2/.3/.4/.5/.7/.13/.14/.24/.27/.39/. gate-result event emitters; integrates with the consuming project's CI v
HRD-022	task	open	middle	

ID	Type	Status	Criticality	Title
				rherald release bindings (§4 tag mirroring, §5 changelog, §11.4.38 evidence, §11.4.40 full-suite retest)
HRD-023	task	open	middle	pherald project bindings (§2 commit+push, §3 submodule propagation §11.4.11/.15/.21/.22/.34/.37/.42/.55/.66/.71/.74)
HRD-024	task	open	middle	iherald constitution-rule escalation bindings (§11.4.10/.10.A credentials §11.4.21 + §11.4.66 operator-blocked escalation)
HRD-025	task	open	low	scherald scheduled-audit bindings (§11.4.45 periodic Status.md sync monthly compliance digest)
HRD-026	task	open	middle	Constitution-bundle hash captureer — computes SHA-256 of rendered evaluation time; persists on every emitted event for replayability
HRD-027	task	open	middle	Mode-ladder runtime config (constitution_bindings table + a to flip allow/warn/enforce per binding per tenant without redeploy)
HRD-029	task	open	middle	§2 pherald commit-push — single-entripoint locked commit +
HRD-030	task	open	middle	§3 pherald submodule propagate — owned-submodule wall
HRD-031	task	open	middle	§4 rherald tag mirror — assert tag exists on every owned sub
HRD-032	task	open	low	§5 rherald changelog generate — Conventional Commits - changelogs/<v>.md + multi-format export
HRD-033	task	open	high	§9.1 sherald destructive guard <op> — wrap rm/git-res prerequisite checks
HRD-034	task	open	middle	§9.3 sherald backup snapshot <path> — hardlinked snaps
HRD-035	task	open	middle	§11.4.2 bherald evidence capture <test_id> — capture
HRD-036	task	open	high	§11.4.10 cherald creds scan — gitleaks/trufflehog integration, .credential.leak
HRD-037	task	open	low	§11.4.12 cherald docs sync — regen Issues_Summary / Fixed_ Status_Summary
HRD-038	task	open	low	§11.4.18 cherald script-docs check — assert sibling .md fo
HRD-039	task	open	low	§11.4.19 / .23 cherald fixed align+cherald colorize - + HTML colorizer
HRD-040	task	open	high	§11.4.26 sherald constitution pull — wrap fetch + rebase emits .bundle.updated
HRD-041	task	open	middle	§11.4.27 bherald test-tier verify — 8-tier matrix verificat
HRD-042	task	open	low	§11.4.31 cherald submanifest verify — Submodule-Depen
HRD-043	task	open	high	§11.4.36 pherald install-upstreams — install_upstreams w
HRD-044	task	open	middle	§11.4.37 pherald fetch-guard — pre-edit fetch + rebase enfor
HRD-045	task	open	high	§11.4.40 rherald gate retest — pre-tag full-suite retest gate
HRD-046	task	open	high	§11.4.41 sherald force-push gate — merge-first + per-sessio
HRD-047	task	open	middle	§11.4.45 / .56 scherald status digest — periodic Status.md Status_Summary regen
HRD-048	task	open	low	§11.4.53 cherald fixed-summary sync — standalone Fixed_
HRD-049	task	open	middle	§11.4.55 pherald reopen <HRD-NNN> — Issues→Fixed revers
HRD-050	task	open	middle	§11.4.59 cherald readme sync — README doc-link regen + r
HRD-051	task	open	high	§11.4.60 cherald composite-gate — canonical implementatio COMPOSITE-SYNC
HRD-052	task	open	middle	§11.4.65 cherald export — bulk Markdown export wrapper (md
HRD-053	task	open	high	§11.4.71 pherald pre-push — fetch + investigate + integrate ho

ID	Type	Status	Criticality	Title
HRD-054	task	open	middle	§11.4.73 cherald spec-version check — Revision-vs-edits
HRD-055	task	open	middle	§11.4.74 cherald catalogue-check <pr> — scan PR for Catalogue survey runner over vasic-digital + HelixDevelopment
HRD-056	task	open	high	§12.6 sherald mem-budget watch — daemon-mode 60% threshold .host.safety_breach
HRD-081	task	open	low	Extend digital.vasic.containers/pkg/compose to detect docker compose runtime and either emit runtime-appropriate flags (not compose) or fall back to host-side healthcheck polling. Composes with so compose.WithWait(true) works across both backends. Also parsing for podman-compose ps output (returns 0 services even when not to podman directly). Per §11.4.76: extend upstream submodule, never
HRD-085	task	open	middle	Implement single-task admin operations on commons_infra.pgxTaskRepository: GetByID, Update, Delete. Required for Requeue + MoveToDeadLetter operator tooling. Currently all three return ErrUnsupported (HRD-085). Add one BackgroundTask row by ID; full-row Update including JSONB column; Delete decision per §107.
HRD-086	task	open	middle	Implement worker-side running-task state mutations on commons_infra.pgxTaskRepository — UpdateStatus, UpdateHeartbeat, SaveCheckpoint. Required by River worker reaper. Currently all four return ErrUnsupported (HRD-086). Status guarded by valid-state-machine check; progress as (float64, message); updated_at = now() writeback; checkpoint as opaque []byte alongside the task row.
HRD-087	task	open	low	Implement admin / stats / audit reads on commons_infra.pgxTaskRepository: GetByStatus, GetPendingTasks, CountByStatus, GetTasks. Add the /v1/queue/* introspection endpoints + Peek delegate. Currently all return ErrUnsupported (HRD-087). Scope: status-filtered paginated read slice for Peek; status-bucketed counts; per-task execution-history rows in task_execution_history table.
HRD-088	task	open	middle	Implement queue-recovery reads on commons_infra.pgxTaskRepository: GetStaleTasks, GetByWorkerID. Drives the stuck-task reaper + recovery loop. Currently both return ErrUnsupported (HRD-088). Add threshold filter against updated_at; per-worker assignment lookup when worker disappears.
HRD-089	task	open	low	Implement resource-monitor I/O on commons_infra.pgxTaskRepository: SaveResourceSnapshot, GetResourceSnapshots. Persists resource memory/IO samples for capacity-planning + post-mortem. Currently both return ErrUnsupported (HRD-089). Scope: append-only insert keyed by captured_at; per-task paginated read in reverse-chronological order in task_resource_snapshots table not yet in the §9.6 migration set of this HRD.
HRD-090	task	open	middle	Implement dead-letter handling on commons_infra.pgxTaskRepository: MoveToDeadLetter. Required failure path when a task exhausts retries invariant. Currently returns ErrUnsupported (HRD-090). Scope: move BackgroundTask row to a dead_letter_tasks table with the failed original-row JSONB snapshot; emit a .queue.dead_letter event on commons_constitution per §42.1. Schema target: a dead_letter table not yet in the §9.6 migration set — adding it is part of this HRD.
HRD-131	task	open	middle	Migrate text HRD trackers to versioned SQLite single-source-of-truth (§108.h, 6-phase) — DB committed as authoritative SSoT per operator

ID	Type	Status	Criticality	Title
HRD-122	task	in_progress	middle	Phase 1 (this item): file the scope-locked tracking HRD. Phase 2: scaffold migration Go binary that already lives in the constitution submodule (docs/scripts/workable-items/) — Herald references it per §11.4.7 and never reimplements it. Phase 3: sync md→db (load docs/Issues.mdx/Fixed.md/*_Summary.md/docs/Status.md/docs/CONTINUATION into docs/.workable_items.db). Phase 4: sync db→md byte-identical regeneration so MD[?]DB drift is mechanically impossible). Phase 5: generate *_Summary.md variants + Status/CONTINUATION become derived. Phase 6: prohibit text-direct edits (release-gate guard that the DB is the only Operator divergence from the parent §11.4.93 gitignored-with-regeneration). Herald the DB IS the authoritative artefact — docs/.workable_items.db is controlled (committed), NOT regenerated-on-clone; only the transient (docs/.workable_items.db-wal/-shm) stay gitignored ("Workable items database since it is our single source of truth regarding what we do on the project!"). Scope-lock: this HRD covers Phase 1 (this row) ONLY. Phases 3-6 (md→db sync, db→md byte-identical regeneration, direct-edit prohibition) are follow-on work-items to be opened as Phase 106 spec-change / §107.x evidence invariants are never broken mid-run. §11.4.85 stress+chaos shared scaffold — Go-native commons/stress package (GAP-3 plan §2 / §6 T1). Provides: a bounded concurrent loader — N workers × M iterations of a closure via sync.WaitGroup, no golang.org/x/sync/errgroup is NOT in go.mod so stdlib sync.WaitGroup plan's "prefer NOT adding deps" tooling note); nearest-rank percentile (Percentiles → {p50, p95, p99, max, min, count}) emitted as stress_chaos/<surface>/; and a best-effort host-memory headroom (HostMemHeadroom — vm_stat+sysctl hw.memsize on darwin, meminfo on linux) for the §12.6 60%-ceiling proof (degrades to proof of never a hard failure). Self-tested: 8 unit tests (percentile correctness on dataset, evidence-dir creation, bounded-goroutine proof, host-mem best-effort predicate) PASS under go test -race -count=1. Lowest sensitivity to CLAUDE.md layering (commons L0, importable by every flavor). §11.4.85 Gin /v1/events stress+chaos tests (GAP-3 plan §1 row 1). pherald/internal/http/events_stress_chaos_test.go. POST /v1/events handler (EventsHandler → runner.Run, REAL in-process httptest server dialed by net/http.Client, production Gin middleware chain (cli.TOONMiddleware + auth gateway boundary (PG/Redis/channel via runner.NewFakeRunner) is fake handler, body read+transcode, error→status mapping, and orchestration). STRESS: N=12 workers × M=200 = 2400 CloudEvent POSTs, -race 5xx, all 202; p50=0.706ms/p95=1.755ms/p99=2.665ms/max=5.722ms. CHAOS: (b) duplicate-key-under-load (1200 POSTs, per-worker keys 1188×200, 0 5xx, 0 hang — graceful degrade; (c) input-corruption (7 b truncated JSON, non-UTF8, JSON array, missing-id, garbage, empty) a tagged event_parser: 400, 0 panic, 0 5xx; (d) auth storm — 100 bearer tokens through the REAL commons_auth.GinMiddleware Verifier → 1000×401, 0 bypass (no token minted → no new go.r drop (container pause) → SKIP-with-reason (no runtime; fail-loud-on-hermetically at runner layer HRD-125). Captured evidence under docs/stress-chaos-2026-05-27T132755/stress_chaos/evidence/latency_histogram.csv, throughput.csv, redis_down_fallback.log, category_auth_storm.log, recovery_trace.log, host_memory_headroom.txt, summary
HRD-123	task	in_progress	high	

ID	Type Status	Criticality Title
HRD-124 task	in_progress	(→ HRD-132 confirmed at HTTP layer): a SHARED-key cross-work replay flood triggers the REAL runner . go : 132 CachedRcpt . race — the NewFakeRunner events_processed store caches+shares concurrent replays mutate it (this IS the "activates with Wave 4+ Rece HRD-125 predicted). Test uses per-worker keys (production-faithful: r cache the Receipt today) so the HTTP-surface degrade contract is prov without asserting a property the code cannot honour (= §107 PASS-bl OWNED by HRD-132. Host-mem headroom recorded; HRD-123 is in resource-exhaustion surface) so the §12.6 ceiling gate does not gate it. §11.4.85 Gin / v1 / compliance (cherald) + / v1 / safety_state stress+chaos tests (GAP-3 plan §1 row 2 / §6 T3) — two new test files internal / compliance / compliance_stress_chaos_test . go , each flavor handler (compliance . Handler / safety . Handler) over http test server dialed by net / http . Client , through the produ (cli . TOONMiddleware → auth → handler). Only the external bou §11.4.27: state . NewMemory () (healthy) or errStore { List → drop) for compliance; the auth claims-injector for both; NO production new go . mod dep (toon resolves via the existing commons / cli repl storm fires random bytes so no token is minted). STRESS (each flavor M=200 = 2000 GETs, - race clean — 0 errors, 0 5xx, all 200. Compl p95=2.00ms/p99=2.59ms ~8.4k req/s; safety_state p50=0.49ms/p95=1 ~16k req/s. Content-negotiation correctness under concurrency — alternates Accept : application / toon vs application / json each flavor → 100% Content-Type match to the request's OWN A bleed) AND every toon body is real TOON wire bytes (byte-0 ≠ {/[]. compliance PG-drop (errStore → store-down) → 2000/2000 fail-l body naming the failed dependency (store list failed), 0 fabr swallowed-error 200 { results : [] } would falsely tell an operator ' PG is down — the §107 mode this gate forbids); live container-pause v reason (no runtime; proven hermetically). (a-safety) safety_state has N the hot path by design (Wave 3 §3 — in-memory Aggregator snaps as-5xx variant is honestly SKIP-not-applicable (asserting a non-exister be a reverse-§107 bluff); the code-true fault analogue — concurrent sta load (4 background goroutines hammering UpdateMemPercent+RecordDestructiveOp during 2000 rea read, 0 5xx, - race-clean RWMutex proof — IS exercised instead. (b flavors): 1000 random 32-byte bearer tokens through the REAL commons_auth . GinMiddleware+HMAC(HS256) Verifier - Verified go test - race - count=3 deterministically green across timing-sensitive assertion — the empty-Accept-defaults-to-TOON got shape-check flaky was fixed by pinning Accept : application / inspection paths, per the HRD-123 flaky-oversized-body lesson). Capt docs / qa / HRD - 124 - 20260527T134316 / stress_chaos / { compliance , safety_state } / (latency.json, latency_histogram pg_drop_fail_loud.log / concurrent_mutation.log, auth_storm.log, pg pg_drop_not_applicable.log, summary.md with §12.6 host-mem headr load only (§12/§12.6): small GETs, N≤10 workers, 4 bounded backgro fork/GB-alloc/host-net-change. §11.4.85 Runner 7-stage pipeline stress+chaos tests (GAP-3 plan §1 ro pherald / internal / runner / stress_chaos_test . go exer runner . Run (only the external boundary — PG events_processed st
HRD-125 task	in_progress	§11.4.85 Runner 7-stage pipeline stress+chaos tests (GAP-3 plan §1 ro pherald / internal / runner / stress_chaos_test . go exer runner . Run (only the external boundary — PG events_processed st

ID

Type Status

Criticality Title

HRD-126 task in_progress high

Redis SETNX seam — is faked per §11.4.27; the orchestrator + 7 stag
STRESS: N=16 goroutines × 40 iters concurrent same-key replay + fr
(PASS). CHAOS: (a) 1000× duplicate flood @ 50 parallel under live a
(a') same flood under the real nil-Redis PG-fallback degrade; (b) herm
(SQLSTATE 40P01) injection on the events_processed Insert. Capture
docs/qa/HRD-125-stress-chaos-2026-05-27T131251
stress_chaos/runner/ (latency.json p50=0.252ms/p95=1.075m
histogram CSV, race_clean.log, exactly_once.txt, duplicate_flood.txt, r
deadlock_recovery.log, host_memory_headroom.txt, summary.md). FL
HRD-132): dispatch exactly-once is NOT guaranteed under concu
events_processed archive row is written at Stage 7 (end of pipeline), so
verdict → Stage-7-archive-commit window admits a bounded handful
duplicate dispatches (observed 2–4 of a 1000× flood; never the full 10
(events_processed) IS exactly-once (PG UNIQUE + ON CONFLICT I
assert the honest code-true contract (archival exactly-once + bounded
aspirational sends==1 (that would be a §107 PASS-bluff). A latent r
CachedRcpt.WasReplay data race surfaces only when a store cac
*Receipt (current production PG Lookup returns Receipt=nil → no
activate with the planned Wave 4+ Receipt caching).

§11.4.85 pherald listen inbound-path stress+chaos tests (GAP-3
T5) — two new test files: pherald/internal/inbound/
stress_chaos_test.go (in-package inbound, so it can exercis
extractReplyToMessageID) + pherald/cmd/pherald/
listen_stress_chaos_test.go (drives the REAL runListen
§11.4.27 only the channel boundary (Subscriber) + CC boundary (Coc
faked; the inbound.Dispatcher, its action-routing switch, the ru
orchestration + clean-shutdown path run unmodified. STRESS: (1) N=
concurrent Dispatcher.Handle calls — 0 errors, exactly-once rep
(800/800), goroutine 2→2 (leaked=0), p50=0.41ms/p95=1.64ms/p99=
~25k dispatch/s; (2) 400-event burst through the full runListen loo
clean shutdown on cancel (<3s, SIGTERM analogue), no leak, ~18k e
table-driven corrupt Raw payloads (nil map, missing/nil/string/bool/sli
message_id) → extractReplyToMessageID degrades to (0,e
int32/float64/float32 decode; under a 64-dispatch corrupt-Raw flood
Dispatcher.Handle is panic-free (recover-guard), 0 errors, degra
(no bogus quote id) — HERMETIC CORE; (b) CC subprocess fault —
killed errors.Is-reachable / truncated / marker-less / empty / unknow
tagged inbound: error, NO reply leak, no hang (3s watchdog); (c) o
50 workers × 20 = 1000 redeliveries of the same issue.open collapse
durable sink row (every redelivery reaches the sink: attempts==1000)
analogue of a crash+restart redelivery loop, leaning on the once-only c
proved at the Runner layer; the real-binary SIGKILL+restart variant is
(HERALD_STRESS_LIVE_LISTEN=1 needed: operator Telegram c
pherald). Verified go test -race -count=3 deterministically g
runs on both packages (assertions are value-true: exactly-once reply co
durable row, tagged-error substrings — not timing-sensitive). NO pro
NO new go.mod dep (commons/stresschaos resolves via the e
replace chain). Captured evidence under docs/qa/2026-05-27T1
listen-HRD-126/stress_chaos/listen/ (latency.json, late
throughput.txt, listen_loop_throughput.txt, malformed_payloads.txt,
handle_malformed_raw.txt, cc_fault.txt, once_only_side_effect.txt, ch
process_death_restart.log, hostmem.txt, summary.md). Bounded load c

ID

Type Status

Criticality Title

HRD-127 task in_progress high

$N \leq 50$ goroutines, KiB-scale alloc, in-process context. CancelFunc system process signalled, no fork/GB-alloc/host-net-change.

§11.4.85 claude_code dispatch stress+chaos tests (GAP-3 plan §1 row commons_messaging/dispatch/claude_code/dispatch_stress_chaos_test.go (in-package claude_code) UNEXPORTED buildCmd/bootstrapSession/parseReply) hermetic shims under commons_messaging/dispatch/claude_testdata/fake-claude-*.sh. Per §11.4.27 only the EXTERNAL claude CLI binary — is faked, via those committed shims selected to New(binaryPath, ...) constructor seam (the same seam bootstrap writeFakeClaudeBinary uses); NO production source edited, NO added, NO new go.mod dep (commons/stresschaos resolves via commons replace chain), NO real claude invoked. STRESS: (1) $N=600$ concurrent Dispatch calls (steady-state --resume path, anchored errors, 0 bad-replies, every reply REALLY parsed from the shim's <<< line, every call resumed the SAME pre-seeded UUID (no double-bootstrap drifted; p50=62.9ms/p95=96.4ms/p99=116.9ms/max=147.6ms ~363 d dominated by the per-call shell-shim fork/exec — honest exec routine concurrent COLD-START bootstrap (no anchor) — all dispatches succeed anchor UUID persists, -race clean. CHAOS: (a) process-death — exec (128+SIGKILL, partial stdout) → tagged claude_code: dispatch resume <u>: exit 137: killed mid-write error, no handle partial stdout NOT accepted; + self-kill -KILL \$\$ → tagged error — fake claude sleeps 30s, ctx deadline 800ms → exec.Command fires, Dispatch returns in 803ms (NOT 30s), tagged error; + bootstrap-budget → returns 603ms, anchor NOT persisted on failure); (c) truncated REPLY>>> (half-JSON, exit 0) → parseReply returns explicit decode error, NEVER a silent partial-accept; UNIT corruption taxonomy (truncated half_marker / marker_no_brace / missing_closing_brace / empty) each well-formed reply still parses (discriminating, not blanket-reject). Verify -race -count=3 deterministically green across ALL 3 runs (DATA assertions are value-true: exact parsed-reply fields, tagged-error substr returns — not timing-sensitive). Captured evidence under docs/qa/chaos-2026-05-27T090518/stress_chaos/claude_code/latency_histogram.csv, dispatch_latency.txt, cold_start_bootstrap.txt, subprocess_self_sigkill.log, timeout_cancel.log, bootstrap_timeout_cancelled_reply.txt, race_clean.log, host_memory_headroom.txt, summary

FINDING-1: cold-start concurrent bootstrap is NOT exactly-once (bootstrap.go: 50-52 — last-writer-wins, not serialised, first or bootstrap_count==1 would be a §107 PASS-bluff, honest bound (bootstraps∈[1,workers]) asserted instead; fix direction if ever required the anchor. **FINDING-2: timeout-cancellation latency depends on c process** — production Dispatch wires ctx into exec.CommandC Cancel SIGKILLS only the DIRECT child, and cmd.Output() block is closed by ALL descendants; the real claude is a single long-lived SIGKILL it closes its own stdout and Output() unblocks immediately (fake sleep shim uses exec sleep (single PID, replace-in-place) to n claude ever spawns helper subprocesses that outlive a SIGKILL which cancellation latency would degrade — hardening direction = Cmd.Ca kill (Setpgid + kill(-pgid)); flagged for the dispatcher owner, out of sec test layer. Bounded load only (§12/§12.6): $N \leq 24$ goroutines, each spawn fake-shim child the test owns+reaps; the ONLY process ever killed is a

ID	Type	Status	Criticality	Title
HRD-128	task	in_progress	high	test spawned (self-SIGKILL or ctx-driven kill) — never a real c l a u d fork/GB-alloc/host-net change. claude_code is an in-process exec sur exhaustion surface) so the §12.6 60%-memory ceiling does not gate it HRD-123/HRD-126); host-mem probe recorded for the audit trail. §11.4.85 container-orchestrated / resource-exhaustion stress+chaos tes 6 / §6 T7 + §7 host-safety) — new commons_storage/ resource_stress_chaos_test.go (in-package storage, he tests/test_resource_stress_chaos.sh (LIVE container production source edited, NO new go.mod dep (commons/stress the existing commons → ../commons replace in commons_stor the digital.vasic.database boundary is faked per §11.4.27). (×2, deterministic): (1) disk-full error propagation — a fake datab injects syscall.ENOSPC ("no space left on device") on the migratio the REAL RunMigrations → applyMigration write path (run DDL/INSERT succeed so the fault lands on a real migration body); ass up_exec_attempts>=1 (real write path exercised), error returned error tagged commons_storage: apply + exec up, errors. syscall.ENOSPC) true (wrap chain unbroken, machine-classifiabl reported applied (no partial-success bluff). Captured full error: commo apply v1 (init_core): exec up: write tablespace space left on device. (2) §12.6 host-mem headroom proof - HostMemHeadroom() captured pre/post a 25× in-process disk-full workload's own used_fraction_delta is ≈0 (≈ -0.0003 — host proving the hermetic unit adds negligible host pressure; this is the §12 evidence artefact. OPT-IN (deterministic SKIP without env): real bo fill, HARD-capped at 64 MiB inside os.MkdirTemp with always-run behind HERALD_STRESS_LIVE_DISK=1 (verified runnable + clean in; SKIPS deterministically by default). LIVE container scenarios — (§11.4.3, host-safety-gated): the shell harness owns (a) container OOM memory=<cap> container OOM-killed → confined to container cgr <60% per §12.6) and (b) ≥30s connection-churn against a --memory gated behind HERALD_STRESS_LIVE_OOM=1 + a §12.6 pre-flight that REFUSES to add memory pressure when host used-fraction ≥ 60% would cross 60%). On this run the host was at ~74% (pre-existing unre correctly REFUSED even when opted in — an honest SKIP over an un present but never booted, no herald-oom-* container leaked. Verif -count=3 deterministically green ALL 3 runs (PASS×3 / PASS×3 / S harness×3 deterministic (exit 0, 2 SKIPS, 1 preflight evidence each); g commons_storage package regression-clean. §12 / §12.6 host-safe NO real-host-disk fill, NO host/system process killed, NO systemc t change, NO GB-alloc — the whole point of this unit honoured. Captur docs/qa/HRD-128-resource-2026-05-27T091641/stre resource/ (disk_full_tagged_error.txt, host_memory_headroom.txt, host_memory_preflight.txt, container_oom_confinement.skip.txt, connection_churn_stress.skip.txt, summary.md). §11.4.85 stress+chaos suite wired into the canonical anti-bluff harness e2e_bluff_hunt.sh as new invariants E81-E88 (GAP-3 plan §4. ADDITIVE-ONLY append after E80 (E1-E80 invariant logic untouched are 3 expanded header-comment lines: the Eighty invariants summary, its continuation, and the Exit 0 only when ... E19.. by E1..E88 / E19..E88 text + an E81-E88 description block). Each inv hermetic Go stress/chaos test (no live creds, no container runtime need
HRD-129	task	in_progress	high	

ID

Type Status

Criticality Title

HRD-130 task in_progress high

check helper AND cites the captured-evidence artefact under docs/
stress_chaos/ as its §11.4.2/§11.4.5 positive-evidence anchor thr
sc_anchor helper that greps a SPECIFIC load-bearing value (NOT
a present-but-empty artefact FAILs as a §11.4 PASS-bluff; absent evid
reason like the E71-E80 gate). Mapping: **E81** runner exactly-once-arch
dispatch (HRD-125) → TestRunner_Stress_ConcurrentRep
(-race) + anchor runner/exactly_once.txt:archival_exa
E82 /v1/events chaos (HRD-123) → input-corruption + duplicate
+ anchor events/categorised_errors.txt:all_4xx_no_
malformed body 4xx, never 5xx, never panic); **E83** /v1/complian
(HRD-124) → TestComplianceHTTP_Chaos_PGDropFailsL
compliance/pg_drop_fail_loud.log:fail_loud_no_f
E84 /v1/safety_state (HRD-124) →
TestSafetyHTTP_Chaos_ConcurrentMutationUnderLoa
safety_state/
concurrent_mutation.log:consistent_under_concur
E85 pherald listen inbound (HRD-126) →
TestInbound_Chaos_HandleMalformedRaw_NeverPanic
ExtractReplyToMessageID_Degrades + anchors listen/
handle_malformed_raw.txt:panic_free=1 + listen/
malformed_payloads.txt:all_malformed_degraded_n
claude_code dispatch (HRD-127) → `TestDispatch_Chaos_ProcessDe
§11.4.85 / §1.1 paired mutation gate for the GAP-3 stress+chaos suite
test_stress_chaos_mutation_meta.sh (GAP-3 plan §5 / §
hermetic stress/chaos detector is genuinely LOAD-BEARING: for eve
production resilience/recovery property, MUTATE it → assert the mat
→ restore via git checkout → assert it PASSes again. Modelled b
canonical tests/test_wave6.5_mutation_meta.sh idiom:
pipefail, .git/MUTATION_IN_PROGRESS lockfile (touch first
cleanup_all EXIT INT TERM), dirty-tree + lockfile + pre-exist
pre-flights, run_paired (apply perl mutation → assert marker lande
EXPECT-FAIL → git checkout restore → assert_restored
HEAD → re-run detector EXPECT-PASS), and a final check_quie
asserting no MUTATED SC-M* markers leaked into the four target fil
ONE-AT-A-TIME, FOREGROUND, never backgrounded/concurrent
2026-05-27 concurrent-gate-residue incident memo — it transiently m
source). FIVE paired mutations, each against a hermetic detector (NO
live PG-Redis / real claude needed): **SC-M1** drop the ctx from clau
dispatch.go:92 exec.CommandContext(ctx,...) → exec.
still used by bootstrapSession so it compiles) →
TestDispatch_Chaos_TimeoutContextCancel no longer c
shim at the 800ms deadline → 8s watchdog t.Fatal (FAIL); **SC-M2**
claude_code parseReply decode error (dispatch.go:124-126
DispatchResponse{}, fmt.Errorf("decode reply JS
return resp, nil) → TestDispatch_Chaos_Truncated
shim emits {"...": "truncat — the DECODE path) now returns nil
dispErr == nil { t.Fatalf } (FAIL); **SC-M3** make inbound
dispatcher.go:339 extractReplyToMessageID default
ofreturn 0, fmt.Errorf(...) →
TestInbound_Chaos_ExtractReplyToMessageID_Degra
map cases panic → per-case recover-guard t.Fatalf (FAIL); **SC-M4**
events.go:176-177 mapErrorToStatus event_parser:

ID	Type	Status	Criticality	Title
HRD-132	bug	open	high	http.StatusBadRequest→http.StatusInternalServerError TestEventsHTTP_Chaos_InputCorruption malformed body 100==5 bad5xx counter (FAIL; http.StatusBadRequest still u tenant_resolver: case so no unused-import); SC-M5 swallow t in commons_storage migrator.go:163 applyMigration (re fmt.Errorf("exec up: %w", err)→_ = err; return TestRunMigrations_Chaos_DiskFull_TaggedError EN success → if err == nil { t.Fatalf } silent-swallow guar disclosure — Pass-2 (the mechanical doc): this work-item wrote ONLY Issues row; it did NOT execute the gate, run go test, or edit produc mutation's perl/sed anchor was VERIFIED against the CURRENT pro reading the exact lines (dispatch.go:92 cmd := exec.Comma d.binaryPath;; dispatch.go:124 if err := dec.Deco err != nil {...fmt.Errorf("decode reply JSON: %w err, truncate(after[braceIdx:], 512)); dispatche 0, fmt.Errorf("inbound: Raw.message_id unsupport v); events.go:176 case strings.Contains(msg, "eve :177 return http.StatusBadRequest; migrator.go:16 fmt.Errorf("exec up: %w", err)) — each anchor confirm EXACTLY once (no stale-anchor §11.4 bluff, per the wave4 M2 lessor run-applied to a /tmp COPY (tracked source never touched) where the gofmt -e parsed the mutated Go clean. Pass-5 (the substance): once conductor, one-at-a-time foreground, after the in-flight e2e_bluff_hunt will prove all five stress/chaos detectors are real anti-bluff guards — e production property is removed and PASSes when restored — not gree bash -n clean; chmod +x. Did NOT execute the gate (conductor wil Tighten Runner idempotency window so concurrent same-key replay c dispatch a notification (FINDING from HRD-125 stress/chaos). idempotency.go:64-72 treats a Redis-SETNX-miss-with-no-PC documented §32.1 "Redis-lies-PG-truths" design, to avoid ingest locku events_processed archive row is only written at Stage 7 (OutcomeRecorder.Process) — so concurrent same-key reques Stage-2→Stage-7 window ALL dispatch (observed 2–4 of a 1000× flo 1000). Archival is exactly-once (PG UNIQUE + ON CONFLICT DO dispatch is only bounded → for a notification system this is duplicate- concurrent load. Fix direction: claim the idempotency slot atomically I (INSERT events_processed ... ON CONFLICT DO NOTHING rows-affected as duplicate, so the PG row — not Stage-7 — is the disp the nil-Redis PG-only degrade (HRD-179). ALSO fix the latent runn CachedRcpt.WasReplay data race (mutates a possibly-shared *R today since PG Lookup returns Receipt=nil, activates with Wave 4+ R ship with the §11.4.85 stress test upgraded to assert sends==1 under (the stronger assertion HRD-125 deliberately could NOT make without test-interrupt-on-discovery).

In progress

ID	Type	Status	Criticality	Title	Opened	Last update	References
HRD-008	task	in_progress	middle	Operator-side	2026-05-20	2026-05-20	spec V3 §26.5 — scaffold shipped,

ID	Type	Status	Criticality	Title	Opened	Last update	References
				quickstart compose validation (Postgres + Redis + OTel + pherald container)			live end-to-end run pending operator; Catalogue-Check: no-match (2026-05-20) — bespoke to Herald

Blocked

(none)

Conventions

See [Fixed.md](#) for closed items + Universal §11.4.12/.15/.16/.19/.33/.55/.73/.74 composition rules.